Milestone-2 Report

Smart MCQ Solver Challenge, IIT Madras

Fine-Tuning a Transformer Model for Multiple-Choice Question Answering using DeBERTa-v3

Naini Diwan

Project: Deep Learning and Generative AI

1. Introduction

Multiple-choice question answering (MCQA) requires a model to read a question stem, or prompt, alongside a fixed set of candidate answers, and identify which candidate is correct. This differs from open-ended generation because the model does not need to produce free text: it only needs to rank a small, closed set of options by plausibility. This report documents a solution built for the Smart MCQ Solver Challenge, a Kaggle competition in which each example consists of a prompt and five labelled options (A through E), with exactly one correct answer.

The approach fine-tunes microsoft/deberta-v3-small, a compact variant of DeBERTa (Decoding-enhanced BERT with disentangled attention), using the Hugging Face transformers and Trainer APIs. DeBERTa was chosen because its disentangled attention mechanism, which separately encodes content and position information, has shown strong results on natural language understanding benchmarks relative to its parameter count, making the small variant a reasonable choice when compute and time are constrained.

2. Basic Concepts

2.1 Transformer Encoders and DeBERTa

A transformer encoder processes an input sequence of tokens through stacked layers of self-attention and feed-forward sublayers, producing a contextual representation for each token that depends on the entire sequence rather than a fixed local window. DeBERTa extends the standard BERT-style encoder with two modifications. First, disentangled attention represents each token with two separate vectors, one for its content and one for its relative position, and computes attention weights from the interaction of both, rather than encoding position as a single vector added to the content embedding. Second, an enhanced mask decoder incorporates absolute position information at the output layer, which helps the model recover word order information that disentangled attention alone does not fully capture.

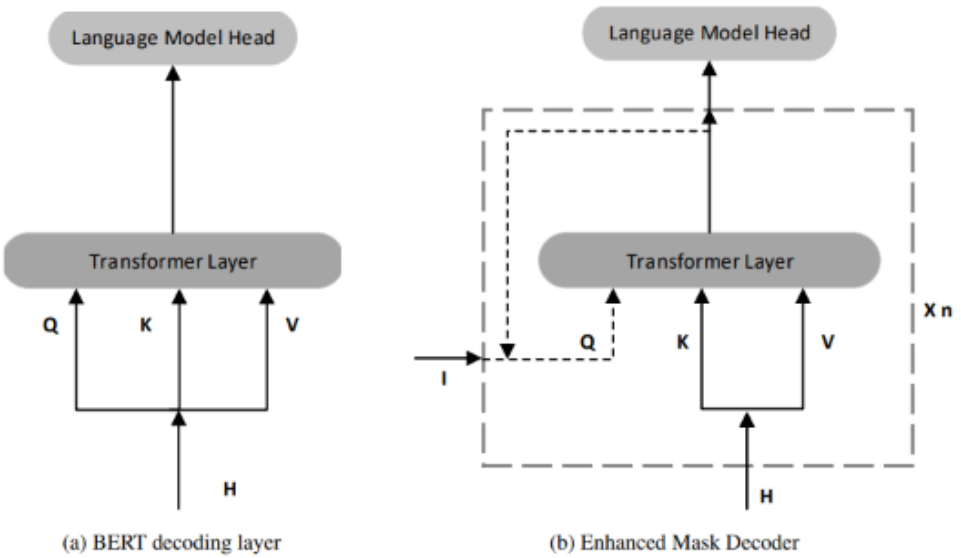

Image courtesy- https://doi.org/10.48550/arXiv.2006.03654

2.2 Multiple-Choice Modelling with AutoModelForMultipleChoice

The Hugging Face AutoModelForMultipleChoice head reframes multiple-choice answering as a scoring problem. For a question with N candidate options, the prompt is paired separately with each option, producing N independent input sequences. Each sequence is passed through the encoder, and a linear classification layer maps the pooled representation of each sequence to a single scalar score. The N scores are then treated as logits over the N choices and normalised with a softmax, so the model is effectively trained with a cross-entropy loss over which of the N options is correct. This head is not present in a general-purpose pretrained checkpoint such as deberta-v3-small, so its weights are randomly initialised and learned entirely during fine-tuning, while the encoder weights start from the pretrained checkpoint.

2.3 Tokenization and Batch Shape

Each of the five (prompt, option) pairs for a question is tokenized independently with padding to a fixed maximum length and truncation for longer inputs. The resulting tensors are stacked so that a single training example has shape [num_choices, sequence_length] rather than the [sequence_length] shape used in standard single-sequence classification. A batch therefore has shape [batch_size, num_choices, sequence_length], and a custom data collator is used to stack these correctly rather than relying on the default collator, which does not handle this extra dimension.

3. Methodology

3.1 Data

Training and test data are read from train.csv and test.csv respectively. Each row contains a prompt column, five option columns labelled A through E, and, for the training set, an answer column holding the correct option letter. A fixed random seed (42) is applied to Python, NumPy, and PyTorch random number generators, and to the train/test split itself, so the split and initialization are reproducible across runs.

3.2 Dataset and Collation

HuggingFaceMCQDataset, builds one training example per question. For each row, it constructs five (prompt, option) text pairs and tokenizes them together using the DeBERTa tokenizer, with padding to a maximum length and truncation of longer sequences. input_ids and attention_mask tensors have shape [5, max_length]. When not operating in test mode, the dataset also attaches an integer label derived from mapping the letter in the answer column (A–E) to an index (0–4). A companion collate function, mcq_data_collator, stacks thesetensors into batches of shape [batch_size, 5, max_length].

3.3 Model Configuration

The base checkpoint is microsoft/deberta-v3-small, loaded through AutoModelForMultipleChoice. Because this checkpoint was pretrained with a masked-language-modelling objective rather than a multiple-choice objective, several weights, such as the language-model prediction head, are unused and discarded on load, while a new pooler and classification head are randomly initialised.

Hyperparameter	Value
Base checkpoint	microsoft/deberta-v3-small
Max sequence length	128 tokens
Number of choices	5 (A–E)
Epochs	3
Learning rate	1e-5
LR scheduler	Cosine, 10% warmup
Max gradient norm	0.5
Train / eval batch size	8 / 8
Gradient accumulation steps	2
Effective train batch size	16
Random seed	42

3.4 Training Procedure

Training is orchestrated with the Hugging Face Trainer class, configured through TrainingArguments. Evaluation and checkpoint saving both occur once per epoch, with the best checkpoint (by the configured metric) retained at the end of training and a limit of two saved checkpoints on disk. The learning rate follows a cosine decay schedule with a 10 percent warmup period, and gradients are clipped to a maximum norm of 0.5 to guard against unstable updates. Gradient accumulation over two steps combines with a per-device batch size of 8 to give an effective training batch size of 16. Training and evaluation metrics are logged to Weights and Biases for run tracking.

3.5 Inference and Submission

After training, the best checkpoint is set to evaluation mode and run over the test set using a DataLoader with batch size 4 and no gradient computation. The five logits produced for each question are converted to probabilities with a softmax, and the three highest-probability options are selected and mapped back to their letter labels. These top-3 predictions are joined into a single space-separated string per question and written, together with the question id, to submission.csv for scoring.

4. Results

The model was evaluated at the end of each of the three training epochs on the 20 percent stratified validation split. Table 2 reports the training loss, validation loss, top-1 accuracy, and macro F1 score logged by the Trainer at each epoch.

Epoch	Training Loss	Validation Loss	Accuracy	F1 (macro)
1	4.695	2.740	0.235	0.232
2	4.430	2.613	0.435	0.439
3	5.199	2.354	0.510	0.511

Validation loss decreases monotonically across all three epochs, from 2.740 to 2.354, and both accuracy and macro F1 improve substantially in step, more than doubling from epoch 1 to epoch 3. The final epoch reaches 51.0 percent top-1 accuracy and a macro F1 of 0.511, which the Trainer selected as the best checkpoint. Since a random classifier over five balanced options would be expected to score approximately 20 percent accuracy, the fine-tuned model performs well above the random baseline after three epochs.

One point worth flagging is that the reported training loss rises between epoch 2 (4.430) and epoch 3 (5.199), even as validation loss and both validation metrics continue to improve over the same interval. Because training loss here is an average over each epoch's mini-batches rather than a smoothed or end-of-epoch snapshot, this pattern can reflect batch-to-batch variance or a small number of high-loss batches rather than genuine overfitting or divergence, particularly since the validation-set trend moves in the opposite, improving, direction. It is nonetheless a detail worth monitoring if training is extended beyond three epochs. No test-set ground truth was available at report time, so the accuracy and F1 figures above reflect validation-split performance only, not leaderboard performance.

5. Conclusion

This report described a fine-tuning pipeline that adapts microsoft/deberta-v3-small to a five-option multiple-choice question answering task using the Hugging Face AutoModelForMultipleChoice head and Trainer API. Over three epochs of full fine-tuning, the model improved from near-random performance to 51.0 percent validation accuracy and a macro F1 of 0.511, with validation loss decreasing consistently throughout training, indicating that the model was still learning productively by the final epoch rather than having plateaued.

Several directions could extend this work. The peft library and LoraConfig are already imported in the notebook but unused; applying LoRA to the encoder layers would let the same experiment be repeated with far fewer trainable parameters, which is worth comparing against the full fine-tuning result reported here on both accuracy and training time. Finally, replacing the single stratified train/validation split with k-fold cross-validation would give a more reliable estimate of generalisation performance than the single-split numbers presented in this report.